AI ENGINEERING · AUTOMOTIVE · LEGACY MODERNIZATION

Your Car Has More Code Than a Fighter Jet.

And it's already obsolete.

> *The software crisis hiding inside every modern vehicle — and why multi-agent AI systems are the only engineering response that scales.*

Let me give you a number that should stop you in your tracks.

A modern premium vehicle runs on **100 to 150 million lines of code**. The F-35 — one of the most sophisticated weapons systems ever built — runs on **5.7 million**. Your car has roughly 25 times more software than a fifth-generation stealth fighter. And yet we still largely maintain it the way we maintained the F-35: with human engineers, working sequentially, one ECU at a time.

Software Scale by Platform

Platform	Lines of Code	Context
Ford F-150 Lightning	**150M**	Modern EV — full drive-by-wire + ADAS stack
Premium Car (avg.)	**100M**	BMW, Mercedes, Audi — typical OEM platform
Boeing 787 Dreamliner	**6.5M**	One of the most complex commercial aircraft
F-35 Fighter Jet	**5.7M**	5th-gen multi-role combat aircraft
SBB Rail Mainframe	**5–20M**	Legacy COBOL/PL1, retired in 2024 after 60 years

Why Human Engineering Has Already Lost the Race

The automotive software stack is not monolithic. A modern vehicle contains upward of 150 individual Electronic Control Units — each running its own firmware, each with its own failure modes, each coupled to dozens of others through CAN bus, Ethernet, and proprietary middleware. At 100 million lines of code distributed across that architecture, no single engineer, no team, no program office has a complete mental model of the system they are responsible for.

We have crossed a threshold. The codebase is now too large for human cognition to hold. Every new feature — adaptive cruise control, over-the-air update logic, V2X communication stacks — adds tens of thousands of lines of code to a system already beyond comprehension. The attack surface expands. The regression risk compounds. The dependency graph becomes a weapon against the people trying to maintain it.

This is not a resourcing problem. Hiring more engineers does not solve a complexity problem. It creates a coordination problem on top of it.

> *"We don't have a shortage of engineers. We have an architecture of software complexity that has outpaced the cognitive bandwidth of any team you could assemble."*

And the legacy dimension makes this worse. Much of the code running in today's vehicles — and virtually all of the code running in logistics, rail, and heavy transport infrastructure — traces its lineage to systems written in C, Ada, or even COBOL. SBB, the Swiss federal rail operator, only retired its 1960s-era mainframe in 2024. After sixty years. These are not edge cases. They are the load-bearing walls of critical infrastructure.

Introducing MOVA: Multi-Agent Orchestration for Vehicle Architecture

The conventional approach to software modernization is sequential: audit, document, refactor, test, deploy. It is a fundamentally human-shaped process, and it scales linearly at best. For a 100-million-line codebase, linear doesn't cut it.

The paradigm shift is to treat the codebase not as a document to be read and edited, but as an environment to be explored and transformed by a coordinated swarm of specialized AI agents — each with a defined role, each operating in parallel, each feeding outputs into a **shared knowledge graph** that no individual agent (and no individual human) could construct alone.

I call this pattern MOVA — Multi-agent Orchestration for Vehicle Architecture. It is built on four specialized agent layers:

MAP: Dependency graph construction

Continuous topology mapping across all ECUs and firmware layers. Generates a live graph of every inter-module dependency, API contract, and shared resource.

OBSERVE: Behavioral analysis

Runtime telemetry ingestion and pattern extraction. Identifies anomalous call sequences, performance bottlenecks, and latent failure modes invisible to static analysis.

VALIDATE: Automated compliance

Continuous regression suites, MISRA-C and AUTOSAR compliance checking, and security vulnerability scanning — running in parallel, not sequentially.

ADAPT: Code transformation

Legacy-to-modern transpilation (C → Rust, Ada → C++, COBOL → Java/TypeScript), guided refactoring, and documentation generation at machine speed.

The key architectural insight is the shared knowledge graph. In traditional modernization, each workstream produces siloed artifacts. In MOVA, every agent writes to and reads from a continuously updated graph representing the ground truth of the codebase's structure, behavior, and transformation state. This enables something sequential approaches cannot: *emergent comprehension*. The system as a whole understands what no individual component was designed to understand.

The Numbers That Will Change Budget Conversations

Let me be direct about the business case, because it is not subtle.

Challenge	Human-only approach	MOVA approach
Full codebase audit (100M lines)	3–5 years, 50+ engineers	**Weeks, continuous**
AUTOSAR compliance gap analysis	Manual, sample-based	**Complete, automated**
Regression test coverage	40–60% (resource-limited)	**90%+ (agent-generated)**
Legacy-to-modern translation	Serial, error-prone	**Parallel, validated**
Security vulnerability surface	Spot-checked	**Continuously monitored**

McKinsey estimates that software-related recalls and field updates cost the automotive industry billions annually — a figure that grows proportionally with codebase complexity. Volkswagen, BMW, Mercedes-Benz, and Stellantis are all actively investing in software-defined vehicle platforms precisely because they understand that the software stack is now the primary source of both competitive differentiation and operational risk.

The Agentic Engineering Pipeline Is Not the Future. It's the Present.

OpenAI has already demonstrated agent teams capable of shipping one million lines of production code. Siemens, Bosch, and Continental are deploying AI-assisted validation and simulation at scale. The industrial AI wave is not approaching — it has broken, and the organizations still evaluating it from the shore are already behind.

What I find most compelling about the MOVA pattern — and what is underappreciated in current industry conversations — is that it reframes the role of the software engineer entirely. The engineer's value in an agentic pipeline is not in writing or reading code at scale. It is in **designing the constraints** under which agents operate: the safety boundaries, the validation criteria, the escalation thresholds, the risk taxonomy. This is fundamentally a risk management function.

The engineers who will thrive in the next decade of automotive software are not those who can read a 100-million-line codebase. No one can. They are those who can ***design the agents that do exactly that***.

A Multi-Agent AI system for maintaining legacy code on edge systems (cars, trains, planes, ships, etc.)

This idea is gold, but only if we design it with extreme realism about the constraints of safety-critical, long-lived, resource-starved embedded worlds.

Where multi-agent systems shine for legacy edge code

1. **Division of labour at massive scale**
2. You can have specialised agents that actually mirror the physical architecture:

a. **Comprehension Agent** – reverse-engineers undocumented C/assembly codebases (often 20–40 years old) and builds a living knowledge graph of every ECU, its CAN matrix, timing constraints, and failure modes.
b. **Telemetry & Anomaly Agent** – lives in the cloud or on a gateway and watches real-world CAN traces + sensor data from thousands of vehicles. It spots emerging failure patterns before they become recalls.
c. **Simulation & Verification Agent** – runs hardware-in-the-loop (HIL) or virtual ECUs at 100× real-time speed to test proposed changes against millions of scenarios (including corner cases that human test engineers miss).
d. **Compliance & Safety Agent** – checks every proposed patch against ISO 26262, DO-178C, IEC 61508, etc., and generates the required traceability, fault-tree analysis, and formal proofs.
e. **Update Orchestration Agent** – plans safe OTA rollouts, rollback strategies, and A/B testing across fleets while respecting regulatory "type approval" rules.

Because the agents can argue with each other (debate-style prompting or even a small "critique loop"), the overall system becomes far more reliable than a single monolithic LLM.

3. Legacy code is the perfect use case

Most automotive, aerospace and marine codebases are huge, poorly documented, and written in languages LLMs already understand well (C, C++, Ada, even assembler). The real bottleneck today is not generation — it's understanding the undocumented side-effects and the implicit assumptions baked into 150 interacting ECUs. Multi-agent setups excel exactly at that kind of distributed reasoning.

4. Edge-native constraints actually help

You don't run the heavy agents on the vehicle. You keep a tiny "edge sentinel" (a few MB of code) that only collects CAN traces, logs, and health metrics and ships them to the cloud agents when connectivity exists (or via satellite for ships/planes). The vehicle itself stays deterministic and certifiable. This is already how Tesla, Rivian, and modern aircraft do OTA today — just supercharge it with agents.

The hard realities (where I'm pessimistic if ignored)

- **Certification & liability** – Regulators will not let an AI agent push code to a flight-control ECU or a brake ECU without human sign-off and reproducible formal verification. So the system must produce artefacts that a certification authority can audit (not just "trust me, the agent said it's safe").
- **Real-time and determinism** – Legacy CAN networks have strict timing; an AI that suggests a 2 µs timing change can brick an entire vehicle. The simulation agent must run at cycle-accurate level.
- **Air-gapped or low-bandwidth fleets** – Many ships, trains, and older aircraft have no continuous internet. The agents must work in "batch mode" with USB sticks or satellite bursts.

- **IP and safety culture** – OEMs and Tier-1 suppliers are paranoid about sharing code. You need federated or on-premise agent deployments.

Realistic deployment path I would actually bet on

Phase 1 (now–2 years): Assistant swarm for human engineers

→ Agents help with root-cause analysis, test-case generation, and documentation. Already huge ROI.

Phase 2 (2–5 years): Supervised autonomous patches

→ Agents propose OTA patches for non-safety-critical ECUs (infotainment, HVAC, body control) with human + automated verification loop.

Phase 3 (5+ years): Full multi-agent closed-loop maintenance for entire fleets

→ Only once the compliance agent can produce certifiable artefacts and regulators accept AI-assisted evidence (we're already seeing FAA/EASA pilots for ML-based certification tools).

Bottom line: Multi-agent AI is probably the only scalable way to keep the exploding number of legacy ECUs alive as vehicles and aircraft stay in service for 15–30 years. A single human team cannot keep up with 150+ firmware variants across millions of units. But the agents must be built like the systems they maintain — modular, verifiable, and paranoid about safety — not like today's flashy chatbot swarms.

If xAI or anyone else builds this with formal verification baked in from day one, I think it becomes one of the highest-leverage applications of AI in the physical world. The CAN bus quote in the question is a perfect reminder why: the software isn't one big blob you can refactor; it's a living, distributed, fault-tolerant network of 150+ independent brains. Only a multi-agent system has the right shape to keep that network healthy for decades.